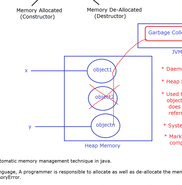
```
package com.ravi.static_factory_method;

import java.util.Scanner;

public class EmployeeDemo
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        IO.print("Enter Employee Id :");
        int id = Integer.parseInt(sc.nextLine());

        IO.print("Enter Employee Name :");
        String name = sc.nextLine();

        IO.print("Enter Employee Salary :");
        double salary = Double.parseDouble(sc.nextLine());

        Employee obj = Employee.getEmployeeObject(id, name, salary);
        IO.println(obj);
        sc.close();

    }

}

class Employee
{
    private int id;
    private String name;
    private double salary;

    private Employee(int id, String name, double salary)
    {
        super();
        this.id = id;
        this.name = name;
        this.salary = salary;
    }

    //static factory Method
    public static Employee getEmployeeObject(int id , String name, double salary)
    {
        //from here we need to create & return the Employee Object
        Employee emp = new Employee(id, name, salary);
        return emp;
    }


    @Override
    public String toString()
    {
        return "Employee [id=" + id + ", name=" + name + ", salary=" + salary + "]";
    }

}
```

Note : By using getEmployeeObject() method we can recieve the Employee object but the main purpose of method is reusability that means we can call the method multiple times to get multiple objects as shown in the program below.

```
package com.ravi.static_factory_method;

import java.util.Scanner;

public class ProductDemo
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        IO.println("How many objects you want :");
        int noOfObj = Integer.parseInt(sc.nextLine());

        for(int i=1; i<=noOfObj; i++)
        {
            System.out.print("Enter product Id :");
            int id = Integer.parseInt(sc.nextLine());

            System.out.print("Enter product Name :");
            String name = sc.nextLine();

            System.out.print("Enter product Price :");
            double price = Double.parseDouble(sc.nextLine());

            Product obj = Product.getProductObject(id, name, price);
            System.out.println("Original Data :"+obj);
            obj.calculateGST();
            System.out.println("After adding GST :"+obj);
        }
    }
}
class Product
{
    private int id;
    private String name;
    private double price;

    private Product(int id, String name, double price)
    {
        super();
        this.id = id;
        this.name = name;
        this.price = price;
    }

    @Override
    public String toString()
    {
        return "Product [id=" + id + ", name=" + name + ", price=" + price + "]";
    }

    public void calculateGST()
    {
        double gstAmount = 0.0;

        if(this.price>=75000)
        {
            gstAmount = this.price * 0.18;
        }
        else if(this.price >=50000)
        {
            gstAmount = this.price * 0.12;
        }
        else
        {
            gstAmount = this.price * 0.05;
        }
        this.price = this.price + gstAmount;
    }


    //static factory method
    public static Product getProductObject(int id, String name, double price)
    {
        return new Product(id, name, price);
    }

}
```

Non static metods are used to perform some operation on the objects.

**** Garbage Collector :**



* Daemon thread
* Heap Memory size is 60 - 65 %
* Used to delate the un-used objects (the object which does not contain any references)
* System.gc();
* Mark & sweep, Mark & compact

It is an automatic memory management technique in java.

In C++ language, A programmer is responsible to allocate as well as de-allocate the memory otherwise we will get OutOfMemoryError.

In Java language, Programmer is only responsible to allocate the memory, Memory de-allocation is automatically done by garbage collector.

Garbage Collector is a **daemon thread** which is responsible to delete the un-used objects from the HEAP Memory. Actually, It scans the heap memory and identifying which objects are eligible for Garbage Collector. (THE OBJECTS WHICH DOES NOT CONTAIN ANY REFERENCES, ONLY THOSE OBJECTS ARE ELIGIBLE FOR GC)

When the size of HEAP memory is around 60-65% then JVM will make a request to the garbage collector to visit the heap memory to delete the un-used objects.

It internally uses algorithms called **Mark and Sweep, Mark and Compact** algorithm to delete the un-used objects.

As a developer we can also explicitly call garbage collector by writing the following code

System.gc(); //Calling Garbage collector explicitly